

With multibyte fields, the least significant byte of the field starts with the lowest byte offset, and subsequent bytes are strictly increasing in significance. I.e., this is little endian format within each multibyte field as well as the overall payload.

3.1.11.8 Route Table Update Response (RTUpdateAck, RTUpdateNak) VDMs

The response to the RTUpdate VDM shall be one of the following:

- RTUpdateAck VDM if the update is successful
- RTUpdateNak VDM if the update is unsuccessful
- RTUpdateNak VDM if a VDM in the sequence was lost

The DPID is set to the vUSP's fabric port's PID, which routes the RTUpdateAck VDM back to the Downstream ES. However, the Downstream ES ingress shall trap on the CXL VDM Code of A1h to direct the VDM to switch firmware.

The Downstream ES, upon receipt of the RTUpdateAck VDM, shall set the commit complete bit in the CacheID table.

The 'RTUpdateAck' / 'RTUpdateNak' VDM fields are as follows. PTH holds:

- SPID = Host Edge Port PID
- DPID = vUSP's fabric port's PID
- DSAR flag = 1

VDM header fields for 'RTUpdateAck' / 'RTUpdateNak' Response VDMs are as follows:

- CXL VDM Code of A1h
- PBR Opcode 12h (RTUpdateAck) / PBR Opcode 13h (RTUpdateNak)

3.2 CXL.cache

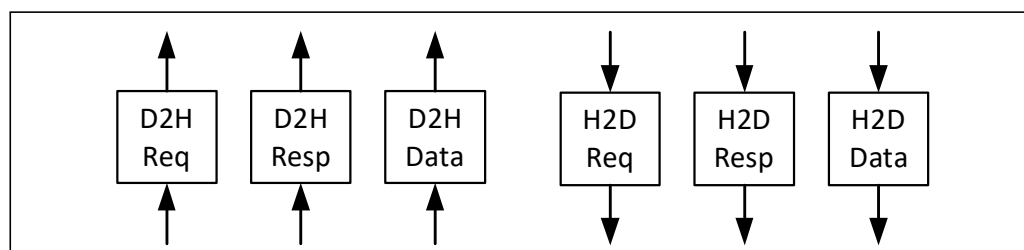
3.2.1 Overview

The CXL.cache protocol defines the interactions between the device and host as a number of requests that each have at least one associated response message and sometimes a data transfer. The interface consists of three channels in each direction: Request, Response, and Data. The channels are named for their direction, D2H for device to host and H2D for host to device, and the transactions they carry, Request, Response, and Data as shown in [Figure 3-10](#). The independent channels allow different kinds of messages to use dedicated wires and achieve both decoupling and a higher effective throughput per wire.

A D2H Request carries new requests from the Device to the Host. The requests typically target memory. Each request will receive zero, one, or two responses and at most one 64-byte cacheline of data. The channel may be back pressured without issue. D2H Response carries all responses from the Device to the Host. Device responses to snoops indicate the state the line was left in the device caches, and may indicate that data is being returned to the Host to the provided data buffer. They may still be blocked temporarily for link layer credits. D2H Data carries all data and byte enables from the Device to the Host. The data transfers can result either from implicit (as a result of snoop) or explicit write-backs (as a result of cache capacity eviction). A full 64-byte cacheline of data is always transferred. D2H Data must make progress or deadlocks may occur. D2H Data may be temporarily blocked for link layer credits, but must not require any other D2H transaction to complete to free the credits.

An H2D Request carries requests from the Host to the Device. These are snoops to maintain coherency. Data may be returned for snoops. The request carries the location of the data buffer to which any returned data should be written. H2D Requests may be back pressured for lack of device resources; however, the resources must free up without needing D2H Requests to make progress. H2D Response carries ordering messages and pulls for write data. Each response carries the request identifier from the original device request to indicate where the response should be routed. For write data pull responses, the message carries the location where the data should be written. H2D Responses can only be blocked temporarily for link layer credits. H2D Data delivers the data for device read requests. In all cases a full 64-byte cacheline of data is transferred. H2D Data transfers can only be blocked temporarily for link layer credits.

Figure 3-10. CXL.cache Channels



3.2.2 CXL.cache Channel Description

3.2.2.1 Channel Ordering

In general, all the CXL.cache channels must work independently of one another to ensure that forward progress is maintained. For example, because requests from the device to the Host to a given address X will be blocked by the Host until it collects all snoop responses for this address X, linking the channels would lead to deadlock.

However, there is a specific instance where ordering between channels must be maintained for the sake of correctness. The Host needs to wait until Global Observation (GO) messages, sent on H2D Response, are observed by the device before sending subsequent snoops for the same address. To limit the amount of buffering needed to track GO messages, the Host assumes that GO messages that have been sent over CXL.cache in a given cycle cannot be passed by snoops sent in a later cycle.

For transactions that have multiple messages on a single channel with an expected order (e.g., WritePull and GO for WrInv) the Device/Host must ensure they are observed correctly using serializing messages (e.g., the Data message between WritePull and GO for WrInv as shown in Figure 3-14).

3.2.2.2 Channel Crediting

To maintain the modularity of the interface no assumptions can be made on the ability to send a message on a channel because link layer credits may not be available at all times. Therefore, each channel must use a credit for sending any message and collect credit returns from the receiver. During operation, the receiver returns a credit whenever it has processed the message (i.e., freed up a buffer). It is not required that all credits are accounted for on either side, it is sufficient that credit counter saturates when full. If no credits are available, the sender must wait for the receiver to return one.

Table 3-12 describes which channels must drain to maintain forward progress and which can be blocked indefinitely. Additionally, Table 3-12 defines a summary of the forward progress and crediting mechanisms in CXL.cache, but this is not the complete definition. See Section 3.4 for the complete set of the ordering rules that are required for protocol correctness and forward progress.

Table 3-12. CXL.cache Channel Crediting Summary

Channel	Forward Progress Condition	Blocking Condition	Description
D2H Request (Req)	Credited to Host	Can be blocked by all other message classes in CXL.cachemem.	Needs Host buffer, could be held by earlier requests
D2H Response (Rsp)	Pre-allocated	Temporary link layer back pressure is allowed. Host may block waiting for H2D Response to drain.	Headed to specified Host buffer
D2H Data	Pre-allocated	Temporary link layer back pressure is allowed. Host may block for H2D Data to drain.	Headed to specified Host buffer
H2D Request (Req)	Credited to Device	Must make progress. Temporary back pressure is allowed.	May be back pressured temporarily due to lack of availability of D2H Response or D2H Data credits
H2D Response (Rsp)	Pre-allocated	Link layer only, must make progress. Temporary back pressure is allowed.	Headed to specified device buffer
H2D Data	Pre-allocated	Link layer only, must make progress. Temporary back pressure is allowed.	Headed to specified device buffer

3.2.3 CXL.cache Wire Description

The definition of each of the fields for each CXL.cache Channel is provided below. Each message in will support 3 variants: 68B Flit, 256B Flit, and PBR Flit. The use of each of these will be negotiated in the physical layer for each link as defined in Chapter 6.0.

3.2.3.1 D2H Request

Table 3-13. CXL.cache - D2H Request Fields (Sheet 1 of 2)

D2H Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The request is valid.
Opcode	5			The opcode specifies the operation of the request. Details in Table 3-22.
CQID	12			Command Queue ID: The CQID field contains the ID of the tracker entry that is associated with the request. When the response and data are returned for this request, the CQID is sent in the response or data message indicating to the device which tracker entry originated this request. IMPLEMENTATION NOTE CQID usage depends on the round-trip transaction latency and desired bandwidth. A 12-bit ID space allows for 4096 outstanding requests which can saturate link bandwidth for a x16 link at 64 GT/s with average latency of up to 1 us ¹ .
NT	1			For cacheable reads, the NonTemporal bit is used as a hint to indicate to the host how it should be cached. Details in Table 3-14.

Table 3-13. CXL.cache - D2H Request Fields (Sheet 2 of 2)

D2H Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
CacheID	0	4	0	Logical CacheID of the source of the message. Not supported in 68B flit messages. Not applicable in PBR messages where DPID infers this field.
Address[51:6]	46			Carries the physical address of coherent requests.
SPID	0		12	Source PID
DPID	0		12	Destination PID
RSVD	14	7		
Total	79	76	96	

1. Formula assumed in this calculation is: "Latency Tolerance in ns" = "number of Requests" * (64B per Request) / "Peak Bandwidth in GB/s". Assuming a peak bandwidth of 256 GB/s (raw bidirectional bandwidth of a x16 CXL port at 64 GT/s) results in a latency tolerance of 1024 ns.

Table 3-14. Non Temporal Encodings

NonTemporal	Definition
0	Default behavior. This is Host implementation specific.
1	Requested line should be moved to Least Recently Used (LRU) position

3.2.3.2 D2H Response

Table 3-15. CXL.cache - D2H Response Fields

D2H Response	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The response is valid.
Opcode	5			The opcode specifies the what kind of response is being signaled. Details in Table 3-25 .
UQID	12			Unique Queue ID: This is a reflection of the UQID sent with the H2D Request and indicates which Host entry is the target of the response.
DPID	0		12	Destination PID
RSVD	2	6		
Total	20	24	36	

3.2.3.3 D2H Data

Table 3-16. CXL.cache - D2H Data Header Fields

D2H Data Header	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid signal indicates that this is a valid data message.
UQID	12			Unique Queue ID: This is a reflection of the UQID sent with the H2D Response and indicates which Host entry is the target of the data transfer.
ChunkValid	1	0		In case of a 32B transfer on CXL.cache, this indicates what 32-byte chunk of the cacheline is represented by this transfer. If not set, it indicates the lower 32B and if set, it indicates the upper 32B. This field is ignored for a 64B transfer.
Bogus	1			The Bogus bit indicates that the data associated with this evict message was returned to a snoop after the D2H request was sent from the device, but before a WritePull was received for the evict. This data is no longer the most current, so it should be dropped by the Host.
Poison	1			The Poison bit is an indication that this data chunk is corrupted and should not be used by the Host.
BEP	0	1		Byte-Enables Present: Indication that 5 data slots are included in the message where the 5 th data slot carries the 64-bit Byte Enables. This field is carried as part of the Flit header bits in 68B Flit mode.
DPID	0		12	Destination PID
RSVD	1	8		
Total	17	24	36	

3.2.3.3.1 Byte Enables (68B Flit)

In 68B Flit mode, the presence of data byte enables is indicated in the flit header, but only when one or more of the byte enable bits has a value of 0. In that case, the byte enables are sent as a data chunk as described in [Section 4.2.2](#).

3.2.3.3.2 Byte-Enables Present (256B Flit)

In 256B Flit mode, a BEP (Byte-Enables Present) bit is included with the message header that indicates BE slot is included at the end of the message. The Byte Enable field is 64 bits wide and indicates which of the bytes are valid for the contained data.

3.2.3.4 H2D Request

Table 3-17. CXL.cache – H2D Request Fields (Sheet 1 of 2)

H2D Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid signal indicates that this is a valid request.
Opcode	3			The Opcode field indicates the kind of H2D request. Details in Table 3-26 .
Address[51:6]	46			The Address field indicates which cacheline the request targets.

Table 3-17. CXL.cache – H2D Request Fields (Sheet 2 of 2)

H2D Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
UQID	12			Unique Queue ID: This indicates which Host entry is the source of the request.
CacheID	0	4	0	Logical CacheID of the destination of the message. Value is assigned by Switch edge ports and not observed by the device. Host implementation may constrain the number of encodings that the Host can support. Not applicable with PBR messages where DPID infers this field.
SPID	0		12	Source PID
DPID	0		12	Destination PID
RSVD	2	6		
Total	64	72	92	

3.2.3.5 H2D Response

Table 3-18. CXL.cache - H2D Response Fields

H2D Response	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid bit indicates that this is a valid response to the device.
Opcode	4			The Opcode field indicates the type of the response being sent. Details in Table 3-27 .
RspData	12			The response Opcode determines how the RspData field is interpreted as shown in Table 3-27 . Thus, depending on Opcode, it can either contain the UQID or the MESI information in bits [3:0] as shown in Table 3-20 .
RSP_PRE	2			RSP_PRE carries performance monitoring information. Details in Table 3-19 .
CQID	12			Command Queue ID: This is a reflection of the CQID sent with the D2H Request and indicates which device entry is the target of the response.
CacheID	0	4	0	Logical CacheID of the destination of the message. This value is returned by the host based on the CacheID sent in the D2H request. Not applicable with PBR messages where DPID infers this field.
DPID	0		12	Destination PID
RSVD	1	5		
Total	32	40	48	

Table 3-19. RSP_PRE Encodings

RSP_PRE[1:0]	Response
00b	Host Cache Miss to Local CPU socket memory
01b	Host Cache Hit
10b	Host Cache Miss to Remote CPU socket memory
11b	Reserved

Table 3-20. Cache State Encoding for H2D Response

Cache State	Encoding
Invalid (I)	0011b
Shared (S)	0001b
Exclusive (E)	0010b
Modified (M)	0110b
Error (Err) ¹	0100b

1. Covers error conditions not covered by poison such as errors in coherence resolution.

3.2.3.6 H2D Data

Table 3-21. CXL.cache - H2D Data Header Fields

H2D Data Header	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid bit indicates that this is a valid data to the device.
CQID	12			Command Queue ID: This is a reflection of the CQID sent with the D2H Request and indicates which device entry is the target of the data transfer.
ChunkValid	1	0		In case of a 32B transfer on CXL.cache, this indicates what 32-byte chunk of the cacheline is represented by this transfer. If not set, it indicates the lower 32B and if set, it indicates the upper 32B. This field is ignored for a 64B transfer.
Poison	1			The Poison bit indicates to the device that this data is corrupted and as such should not be used.
GO-Err	1			The GO-ERR bit indicates to the agent that this data is the result of an error condition and should not be cached or provided as response to snoops. Covers error conditions not covered by poison such as errors in coherence resolution.
CacheID	0	4	0	Logical CacheID of the destination of the message. Host and switch must support this field to set a nonzero value. Not applicable in PBR messages where DPID infers this field.
DPID	0		12	Destination PID
RSVD	8	9		
Total	24	28	36	

3.2.4 CXL.cache Transaction Description

3.2.4.1 Device-attached Memory Flows for HDM-D/HDM-DB

When a CXL Type 2 device exposes memory to the host using Host-managed Device Memory Device-Coherent (HDM-D/HDM-DB), the device is responsible to resolve coherence of HDM between the host and device. CXL defines two protocol options for this:

- CXL.cache Requests which is used for HDM-D
- CXL.mem Back-Invalidate Snoop (BISnp) which is used with HDM-DB

Endpoint devices supporting 256B Flit mode must support BISnp mechanism and can optionally use CXL.cache mechanism when connected to a host that has only 68B flit mode. When using CXL.cache, the host detects the address as coming from the device that owns the region which triggers the special flow that returns Mem*Fwd, in most cases, as captured in [Table 3-24](#).

3.2.4.2 Device to Host Requests

3.2.4.2.1 Device to Host (D2H) CXL.cache Request Semantics

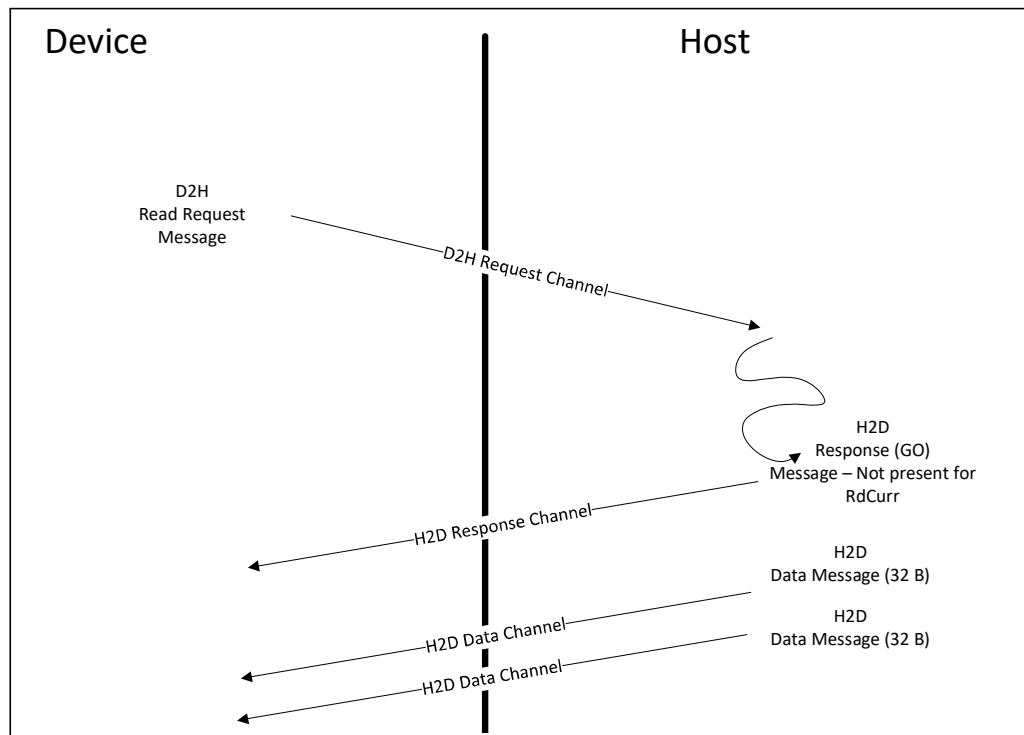
For device to Host requests, there are four different semantics: CXL.cache Read, CXL.cache Read0, CXL.cache Read0/Write, and CXL.cache Write. All device to Host CXL.cache transactions fall into one of these four semantics, though the allowable responses and restrictions for each request type within a given semantic are different.

3.2.4.2.2 CXL.cache Read

CXL.cache Reads must have a D2H request credit and send a request message on the D2H CXL.cache request channel. CXL.cache Read requests require zero or one response (GO) message and data messages totaling a single 64-byte cacheline of data. Both the response, if present, and data messages are directed at the device tracker entry provided in the initial D2H request packet's CQID field. The device entry must remain active until all the messages from the Host have been received. To ensure forward progress, the device must have a reserved data buffer able to accept 64 bytes of data immediately after the request is sent. However, the device may temporarily be unable to accept data from the Host due to prior data returns not draining. Once both the response message and the data messages have been received from the Host, the transaction can be considered complete and the entry deallocated from the device.

[Figure 3-11](#) shows the elements required to complete a CXL.cache Read. Note that the response (GO) message can be received before, after, or between the data messages.

Figure 3-11. CXL.cache Read Behavior

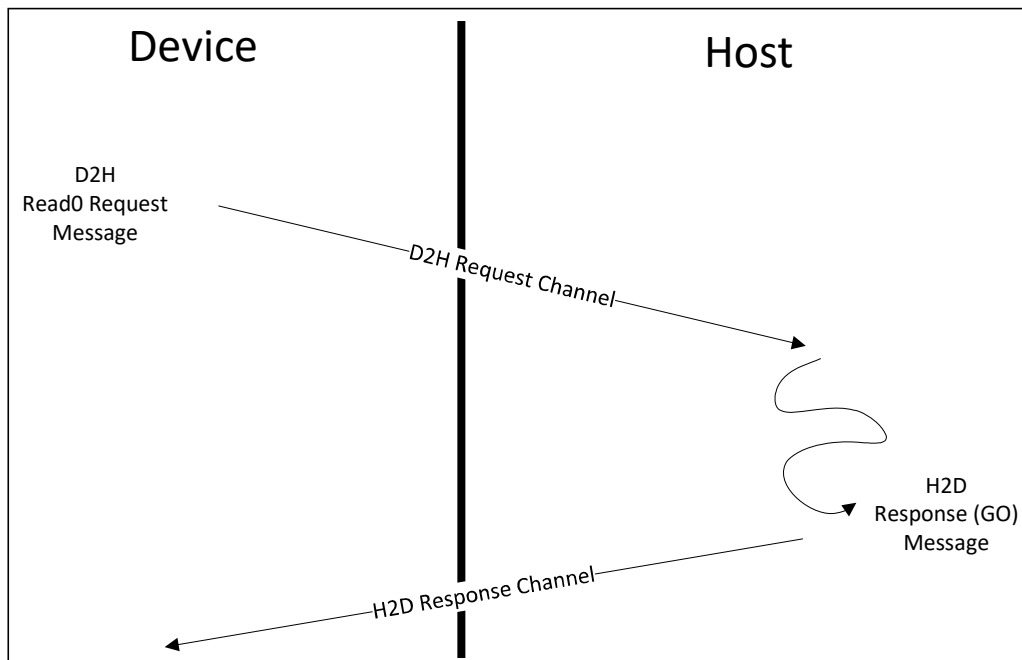


3.2.4.2.3 CXL.cache Read0

CXL.cache Read0 must have a D2H request credit and send a message on the D2H CXL.cache request channel. CXL.cache Read0 requests receive a response message but no data messages. The response message is directed at the device entry indicated in the initial D2H request message's CQID value. Once the GO message is received for these requests, they can be considered complete and the entry deallocated from the device. A data message must not be sent by the Host for these transactions. Most special cycles (e.g., CLFlush) and other miscellaneous requests fall into this category. See [Table 3-22](#) for details.

[Figure 3-12](#) shows the elements required to complete a CXL.cache Read0 transaction.

Figure 3-12. CXL.cache Read0 Behavior



3.2.4.2.4 CXL.cache Write

CXL.cache Write must have a D2H request credit before sending a request message on the D2H CXL.cache request channel. Once the Host has received the request message, it is required to send a GO message and a WritePull message. The WritePull message is not required for CleanEvictNoData. The GO and the WritePull can be a combined message for some requests. The GO message must never arrive at the device before the WritePull, but it can arrive at the same time in the combined message. If the transaction requires posted semantics, then a combined GO-I/WritePull message can be used. If the transaction requires non-posted semantics, then WritePull is issued first followed by the GO-I when the non-posted write is globally observed.

Upon receiving the GO-I message, the device will consider the store done from a memory ordering and cache coherency perspective, giving up snoop ownership of the cacheline (if the CXL.cache message is an Evict).

The WritePull message triggers the device to send data messages to the Host totaling exactly 64 bytes of data, though any number of byte enables can be set.

A CXL.cache write transaction is considered complete by the device once the device has received the GO-I message, and has sent the required data messages. At this point the entry can be deallocated from the device.

The Host considers a write to be done once it has received all 64 bytes of data, and has sent the GO-I response message. All device writes and Evicts fall into the CXL.cache Write semantic.

See [Section 3.2.5.8](#) for more information on restrictions around multiple active write transactions.

Figure 3-13 shows the elements required to complete a CXL.cache Write transaction (that matches posted behavior). The WritePull (or the combined GO_WritePull) message triggers the data messages. There are restrictions on Snoops and WritePulls. See Section 3.2.5.3 for more details.

Figure 3-14 shows a case where the WritePull is a separate message from the GO (for example: strongly ordered uncacheable write).

Figure 3-15 shows the Host FastGO plus ExtCmp responses for weakly ordered write requests.

Figure 3-13. CXL.cache Device to Host Write Behavior

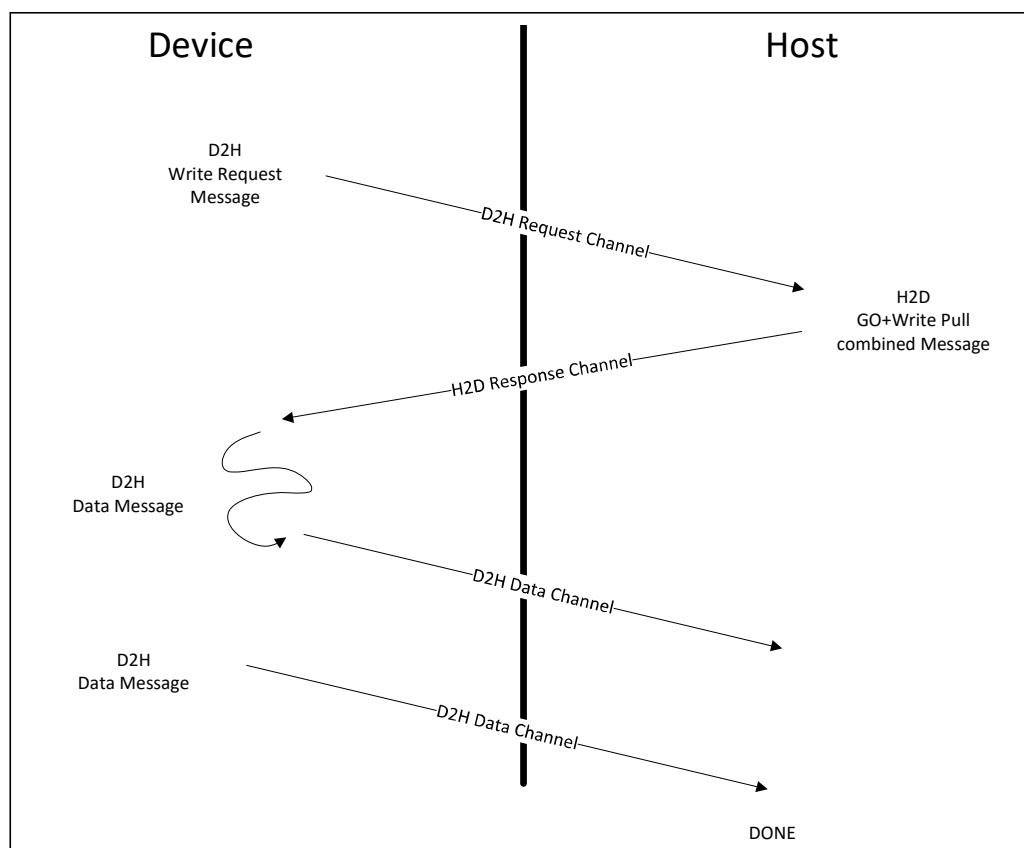


Figure 3-14. CXL.cache WrInv Transaction

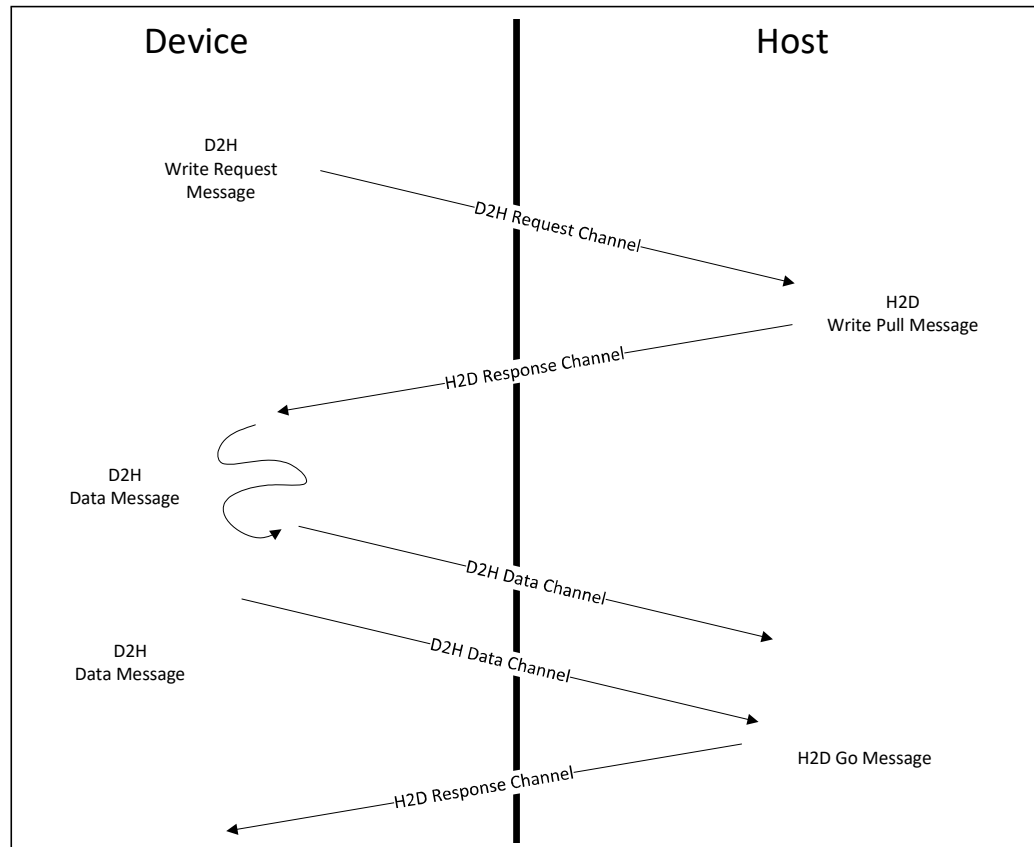
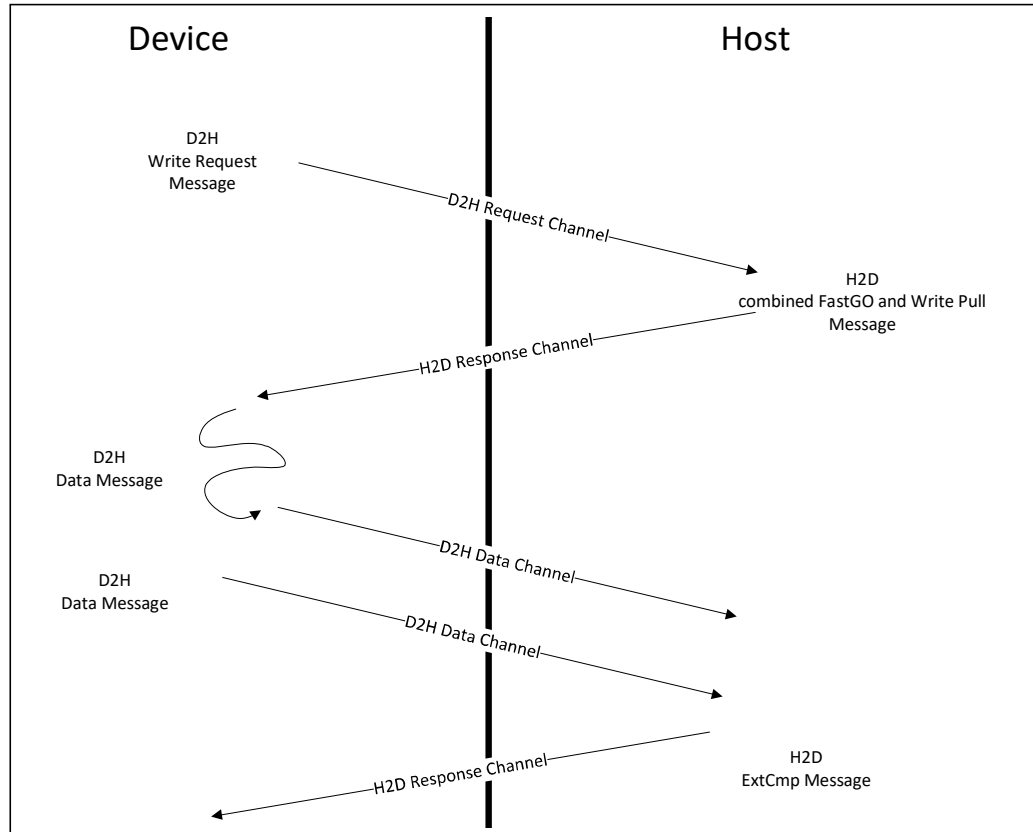


Figure 3-15. WOWrInv/F with FastGO/ExtCmp



3.2.4.2.5 CXL.cache Read0-Write Semantics

CXL.cache Read0-Write requests must have a D2H request credit before sending a request message on the D2H CXL.cache request channel. Once the Host has received the request message, it is required to send one merged GO-I and WritePull message.

The WritePull message triggers the device to send the data messages to the Host, which together transfer exactly 64 bytes of data though any number of byte enables can be set.

A CXL.cache Read0-Write transaction is considered complete by the device once the device has received the GO-I message, and has sent the all required data messages. At this point the entry can be deallocated from the device.

The Host considers a Read0-Write to be done once it has received all 64 bytes of data, and has sent the GO-I response message. ItoMWr falls into the Read0-Write category.

Figure 3-16. CXL.cache Read0-Write Semantics

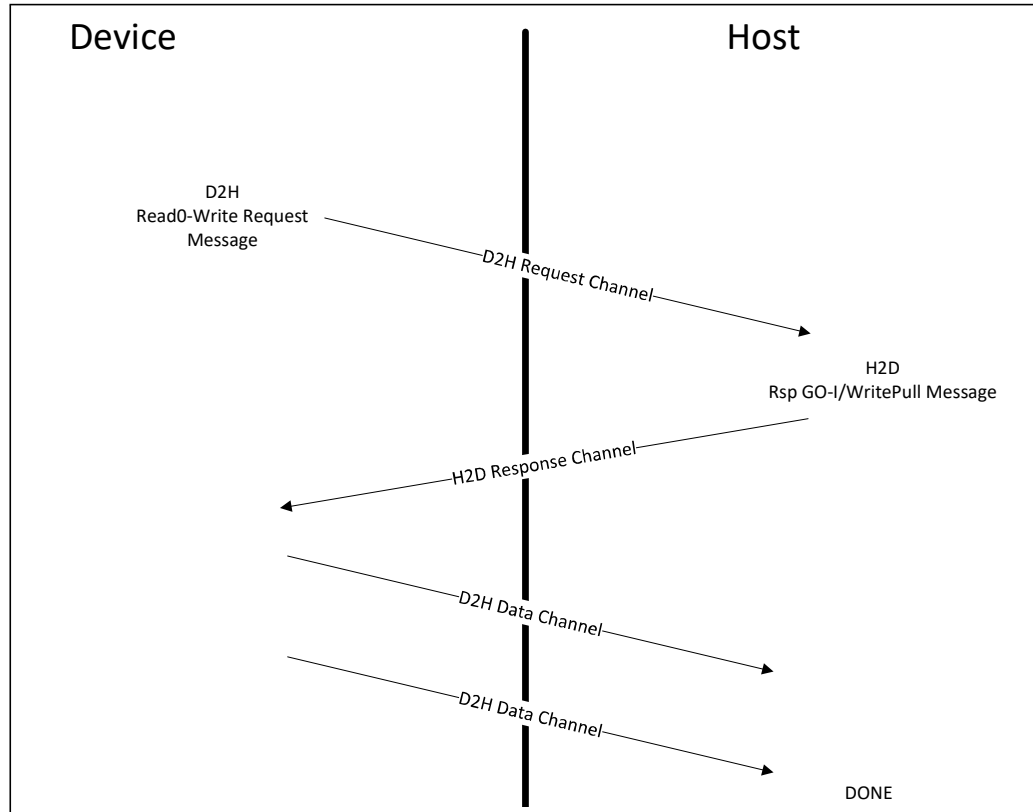


Table 3-22 summarizes all the opcodes that are available from the Device to the Host.

Table 3-22. CXL.cache – Device to Host Requests

CXL.cache Opcode	Semantic	Opcode
RdCurr	Read	0 0001b
RdOwn	Read	0 0010b
RdShared	Read	0 0011b
RdAny	Read	0 0100b
RdOwnNoData	Read0	0 0101b
ItoMWr	Read0-Write	0 0110b
WrCur	Read0-Write	0 0111b
CLFlush	Read0	0 1000b
CleanEvict	Write	0 1001b
DirtyEvict	Write	0 1010b
CleanEvictNoData	Write	0 1011b
WOWrInv	Write	0 1100b
WOWrInvF	Write	0 1101b
WrInv	Write	0 1110b
CacheFlushed	Read0	1 0000b

3.2.4.2.6 RdCurr

These are full cacheline read requests from the device for lines to get the most current data, but not change the existing state in any cache, including in the Host. The Host does not need to track the cacheline in the device that issued the RdCurr. RdCurr gets a data but no GO. The device receives the line in the Invalid state which means that the device gets one use of the line and cannot cache it.

3.2.4.2.7 RdOwn

These are full cacheline read requests from the device for lines to be cached in any writeable state. Typically, RdOwn request receives the line in Exclusive (GO-E) or Modified (GO-M) state. Lines in Modified state must not be dropped, and have to be written back to the Host.

Under error conditions, a RdOwn request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.8 RdShared

These are full cacheline read requests from the device for lines to be cached in Shared state. Typically, RdShared request receives the line in Shared (GO-S) state.

Under error conditions, a RdShared request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both will return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.9 RdAny

These are full cacheline read requests from the device for lines to be cached in any state. Typically, RdAny request receives the line in Shared (GO-S), Exclusive (GO-E) or Modified (GO-M) state. Lines in Modified state must not be dropped, and have to be written back to the Host.

Under error conditions, a RdAny request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.10 RdOwnNoData

These are requests to get exclusive ownership of the cacheline address indicated in the address field. The typical response is Exclusive (GO-E).

Under error conditions, a RdOwnNoData request may receive the line in Error (GO-Err) state. The device is responsible for handling the error appropriately.

Note:

A device that uses this command to write data must be able to update the entire cacheline or may drop the E-state if it is unable to perform the update. There is no support partial M-state data in a device cache. To perform a partial write in the device cache, the device must read the cacheline using RdOwn before merging with the partial write data in the cache.

3.2.4.2.11 ItoMWr

This command requests exclusive ownership of the cacheline address indicated in the address field and atomically writes the cacheline back to the Host. The device guarantees the entire line will be modified, so no data needs to be transferred to the

device. The typical response is GO_WritePull, which is sent once the request is granted ownership. The device must not retain a copy of the line. If a cache exists in the host cache hierarchy before memory, the data should be written there.

If an error occurs, then GO-Err-WritePull is sent instead. The device sends the data to the Host, which drops it. The device is responsible for handling the error as appropriate.

3.2.4.2.12 WrCur

The command behaves like the ItoMWr in that it atomically requests ownership of a cacheline and then writes a full cacheline back to the Fabric. However, it differs from ItoMWr in where the data is written. Only if the command hits in a cache will the data be written there; on a Miss, the data will be written directly to memory. The typical response is GO_WritePull once the request is granted ownership. The device must not retain a copy of the line.

If an error occurs, then GO-Err-WritePull is sent instead. The device sends the data to the Host, which drops it. The device is responsible for handling the error as appropriate.

Note: In earlier revisions of the specification (CXL 2.0 and CXL 1.x), this command was called "MemWr", but this was a problem because that same message name is used in the CXL.mem protocol, so a new name was selected. The opcode and behavior are unchanged.

3.2.4.2.13 CLFlush

This is a request to the Host to invalidate the cacheline specified in the address field. The typical response is GO-I which is sent from the Host upon completion in memory.

However, the Host may keep tracking the cacheline in Shared state if the Core has issued a Monitor to an address belonging in the cacheline. Thus, the Device that exposes an HDM-D region must not rely on CLFlush/GO-I as a sufficient condition for which to flip a cacheline in the HDM-D region from Host to Device Bias mode. Instead, the Device must initiate RdOwnNoData and receive an H2D Response of GO-E before it updates its Bias Table to Device Bias mode to allow subsequent cacheline access without notifying the Host.

Under error conditions, a CLFlush request may receive the line in the Error (GO-Err) state. The device is responsible for handling the error appropriately.

3.2.4.2.14 CleanEvict

This is a request to the Host to evict a full 64-byte Exclusive cacheline from the device. Typically, CleanEvict receives GO-WritePull or GO-WritePullDrop. The response will cause the device to relinquish snoop ownership of the line. For GO-WritePull, the device will send the data as normal. For GO-WritePullDrop, the device simply drops the data.

Once the device has issued this command and the address is subsequently snooped, but before the device has received the GO-WritePull, the device must set the Bogus field in all D2H Data messages to indicate that the data is now stale.

CleanEvict requests also guarantee to the Host that the device no longer contains any cached copies of this line. Only one CleanEvict from the device may be pending on CXL.cache for any given cacheline address.

CleanEvict is only expected for a host-attached memory range of addresses. For a device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.15 DirtyEvict

This is a request to the Host to evict a full 64-byte Modified cacheline from the device. Typically, DirtyEvict receives GO-WritePull from the Host at which point the device must relinquish snoop ownership of the line and send the data as normal.

Once the device has issued this command and the address is subsequently snooped, but before the device has received the GO-WritePull, the device must set the Bogus field in all D2H Data messages to indicate that the data is now stale.

DirtyEvict requests also guarantee to the Host that the device no longer contains any cached copies of this line. Only one DirtyEvict from the device may be pending on CXL.cache for any given cacheline address.

In error conditions, a GO-Err-WritePull is received. The device sends the data as normal, and the Host drops it. The device is responsible for handling the error as appropriate.

DirtyEvict is only expected for host-attached memory address ranges. For device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.16 CleanEvictNoData

This is a request for the device to update the Host that a clean line is dropped in the device. The sole purpose of this request is to update any snoop filters in the Host and no data is exchanged.

CleanEvictNoData is only expected for host-attached memory address ranges. For device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.17 WOWrInv

This is a weakly ordered write invalidate line request of 0-63 bytes for write combining type stores. Any combination of byte enables may be set.

Typically, WOWrInv receives a FastGO-WritePull followed by an ExtCmp. Upon receiving the FastGO-WritePull the device sends the data to the Host. For host-attached memory, the Host sends the ExtCmp once the write is complete in memory.

FastGO does not provide "Global Observation".

In error conditions, a GO-Err-WritePull is received. The device sends the data as normal, and the Host drops it. The device is responsible for handling the error as appropriate. An ExtCmp is still sent by the Host after the GO-Err in all cases.

3.2.4.2.18 WOWrInvF

Same as WOWrInv (rules and flows), except it is a write of 64 bytes.

3.2.4.2.19 WrInv

This is a write invalidate line request of 0-64 bytes. Typically, WrInv receives a WritePull followed by a GO. Upon getting the WritePull, the device sends the data to the Host. The Host sends GO once the write completes in memory (both, host-attached or device-attached).

In error conditions, a GO-Err is received. The device is responsible for handling the error as appropriate.

3.2.4.2.20 CacheFlushed

This is an indication sent by the device to inform the Host that its caches are flushed, and it no longer contains any cachelines in the Shared, Exclusive or Modified state (a device may exclude addresses that are part its "Device-attached Memory" mapped as HDM-D/HDM-DB). The Host can use this information to clear its snoop filters, block snoops to the device, and return a GO. Once the device receives the GO, it is guaranteed to not receive any snoops from the Host until the device sends the next cacheable D2H Request.

When a CXL.cache device is flushing its cache, the device must wait for all responses for cacheable access before sending the CacheFlushed message. This is necessary because the Host must observe CacheFlushed only after all inflight messages that impact device coherence tracking in the Host are complete.

IMPLEMENTATION NOTE

Snoops may be pending to the device when the Host receives the CacheFlushed command and the Host may complete the CacheFlushed command (sending a GO) while those snoops are outstanding. From the device point of view, this can be observed as receiving snoops after the CacheFlushed message is complete. The device should allow for this behavior without creating long stall conditions on the snoops by waiting for snoop queues to drain before initiating any power state transition (e.g., L1 link state) that could stall snoops.

Table 3-23. D2H Request (Targeting Non Device-attached Memory) Supported H2D Responses

D2H Request	H2D Response										
	WritePull	GO_WritePull	ExtCmp	GO_WritePull_Drop	Fast_GO_WritePull	GO_ERR_WritePull	GO-Err	GO-I	GO-S	GO-E	GO-M
RdCurr											
RdOwn							X	X		X	X
RdShared							X	X	X		
RdAny							X	X	X	X	X
RdOwnNoData							X			X	
ItoMWr		X				X					
WrCur		X				X					
CLFlush							X	X			
CleanEvict		X		X							
DirtyEvict		X				X					
CleanEvictNoData								X			
WOWrInv			X		X	X					
WOWrInvF			X		X	X					
WrInv	X						X	X			
CacheFlushed								X			

For requests that target device-attached memory mapped as HDM-D, if the region is in Device Bias, no transaction is expected on CXL.cache because the Device can internally complete those requests. If the region is in Host Bias, [Table 3-24](#) shows how the device should expect the response. For devices with BISnp channel support in which the memory is mapped as HDM-DB, the resolution of coherence happens separately on the CXL.mem protocol and the “Not Supported” cases in the table are never sent from a device to the device-attached memory address range. The only commands supported on CXL.cache to this address region when BISnp is enabled are ItoMWr, WrCur, and WrInv.

Table 3-24. D2H Request (Targeting Device-attached Memory) Supported Responses

D2H Request	Response on CXL.cache		Response on CXL.mem	
	Without BISnp (HDM-D)	With BISnp (HDM-DB)	Without BISnp (HDM-D)	With BISnp (HDM-DB)
RdCurr	GO-Err Bit set in H2D DH, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdOwn	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdShared	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdAny	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdOwnNoData	GO-Err on H2D Response (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
ItoMWr	Same as host-attached memory ¹		None	
WrCur	Same as host-attached memory ¹		None	
CLFlush	GO-Err on H2D Response (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
CleanEvict	Not Supported			
DirtyEvict	Not Supported			
CleanEvictNoData	Not Supported			
WOWrInv	GO_ERR_WritePull on H2D Response (For Error Conditions)	Not Supported	MemWrFwd (For Success Conditions)	Not Supported
WOWrInvF	GO_ERR_WritePull on H2D Response (For Error Conditions)	Not Supported	MemWrFwd (For Success Conditions)	Not Supported
WrInv	Same as host-attached memory ¹		None	
CacheFlushed	N/A ²		None	

1. Flow for these commands follow the same flow as host memory regions and are not expected to check against CXL.mem coherence tracking (Bias Table or Snoop Filter) before issuing. The host will resolve coherence with the device using the CXL.mem protocol.
2. There is no address in this command and the host must assume that this applies only to host memory regions (excluding device-attached memory).

CleanEvict, DirtyEvict, and CleanEvictNoData targeting device-attached memory should always be completed internally by the device, regardless of bias state. For D2H Requests that receive a response on CXL.mem, the CQID associated with the CXL.cache request is reflected in the Tag of the CXL.mem MemRdFwd or MemWrFwd command. For MemRdFwd, the caching state of the line is reflected in the MetaValue field as described in [Table 3-37](#).

3.2.4.3 Device to Host Response

Responses are directed at the Host entry indicated in the UQID field in the original H2D request message.

Table 3-25. D2H Response Encodings

Device CXL.cache Rsp	Opcode
RspIHitI	0 0100b
RspVHitV	0 0110b
RspIHitSE	0 0101b
RspSHitSE	0 0001b
RspSFwdM	0 0111b
RspIFwdM	0 1111b
RspVFwdV	1 0110b

3.2.4.3.1 RspIHitI

In general, this is the response that a device provides to a snoop when the line was not found in any caches. If the device returns RspIHitI for a snoop, the Host can assume the line has been cleared from that device.

3.2.4.3.2 RspVHitV

In general, this is the response that a device provides to a snoop when the line was hit in the cache and no state change occurred. If the device returns an RspVHitV for a snoop, the Host can assume a copy of the line is present in one or more places in that device.

3.2.4.3.3 RspIHitSE

In general, this is the response that a device provides to a snoop when the line was hit in a clean state in at least one cache and is now invalid. If the device returns an RspIHitSE for a snoop, the Host can assume the line has been cleared from that device.

3.2.4.3.4 RspSHitSE

In general, this is the response that a device provides to a snoop when the line was hit in a clean state in at least one cache and is now downgraded to shared state. If the device returns an RspSHitSE for a snoop, the Host should assume the line is still in the device.

3.2.4.3.5 RspSFwdM

This response indicates to the Host that the line being snooped is now in S state in the device, after having hit the line in Modified state. The device may choose to downgrade the line to Invalid. This response also indicates to the Host snoop tracking logic that 64 bytes of data is transferred on the D2H CXL.cache Data Channel to the Host data buffer indicated in the original snoop's destination (UQID).

3.2.4.3.6 RspIFwdM

This response indicates to the Host that the line being snooped is now in I state in the device, after having hit the line in Modified state. The Host may now assume the device contains no more cached copies of this line. This response also indicates to the Host

snoop tracking logic that 64 bytes of data will be transferred on the D2H CXL.cache Data Channel to the Host data buffer indicated in the original snoop’s destination (UQID).

3.2.4.3.7 RspVFwdV

This response indicates that the device with E or M state (but not S state) is returning the current data to the Host and leaving the state unchanged. The Host must only forward the data to the requester because there is no state information.

3.2.4.4 Host to Device Requests

Snoops from the Host need not gain any credits besides local H2D request credits. The device will always send a Snoop Response message on the D2H CXL.cache Response channel. If the response is of the Rsp*Fwd* format, then the device must respond with 64 bytes of data via the D2H Data channel, directed at the UQID from the original snoop request message. If the response is not Rsp*Fwd*, the Host can consider the request complete upon receiving the snoop response message. The device can stop tracking the snoop once the response has been sent for non-data forwarding cases, or after both the last chunk of data has been sent and the response has been sent.

Figure 3-17 shows the elements required to complete a CXL.cache snoop. Note that the response message can be received by the Host in any relative order with respect to the data messages. The byte enable field is always all 1s for Snoop data transfers.

Figure 3-17. CXL.cache Snoop Behavior

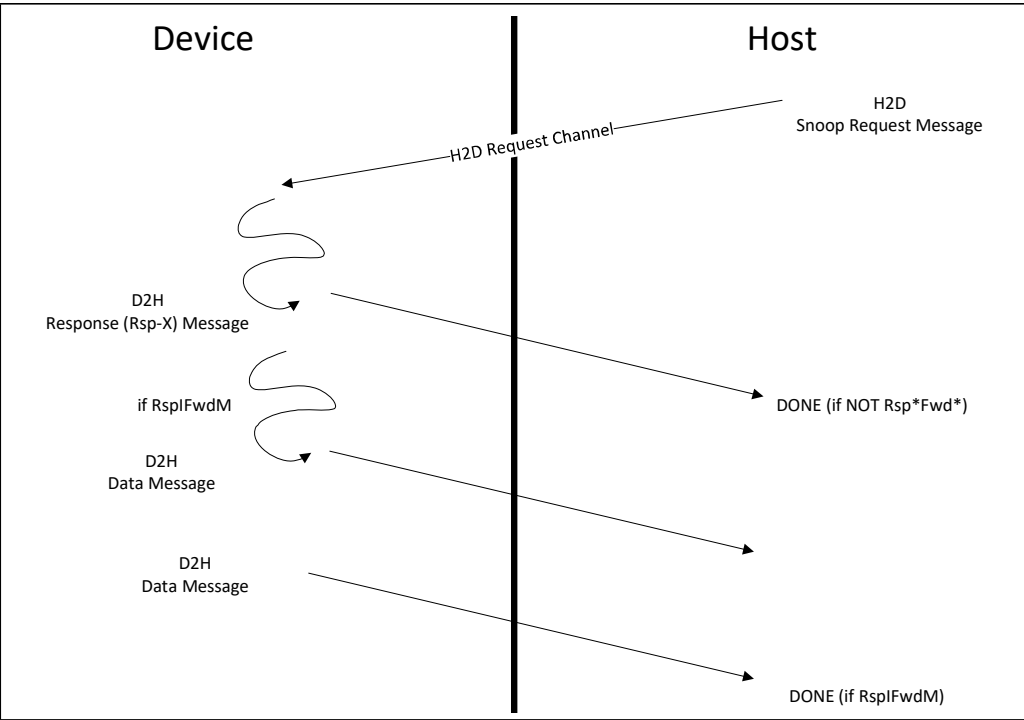


Table 3-26. CXL.cache – Mapping of H2D Requests to D2H Responses

	Opcode	RspIHtI	RspVhitV	RspSHitSE	RspIHtSE	RspSFwdM	RspIFwdM	RspVFwdV
SnpData	001b	X		X		X	X	
SnpInv	010b	X			X		X	
SnpCur	011b	X	X	X		X	X	X

3.2.4.4.1 SnpData

These are snoop requests from the Host for lines that are intended to be cached in either Shared or Exclusive state at the requester (the Exclusive state can be cached at the requester only if all devices respond with RspI). This type of snoop is typically triggered by data read requests. A device that receives this snoop must either invalidate or downgrade all cachelines to Shared state. If the device holds dirty data it must return it to the Host.

3.2.4.4.2 SnpInv

These are snoop requests from the Host for lines that are intended to be granted ownership and Exclusive state at the requester. This type of snoop is typically triggered by write requests. A device that receives this snoop must invalidate all cachelines. If the device holds dirty data it must return it to the Host.

3.2.4.4.3 SnpCur

This snoop gets the current version of the line, but doesn't require change of any cache state in the hierarchy. It is only sent on behalf of the RdCurr request. If the device holds data in Modified state it must return it to the Host. The cache state can remain unchanged in both the device and Host, and the Host should not update its caches. To allow for varied cache implementations, devices are allowed to change cache state as captured in Table 3-26, but it is recommended to not change cache state.

3.2.4.5 Host to Device Response

Table 3-27. H2D Response Opcode Encodings

H2D Response Class	Encoding	RspData
WritePull	0001b	UQID
GO	0100b	MESI ¹
GO_WritePull	0101b	UQID
ExtCmp	0110b	Don't Care
GO_WritePull_Drop	1000b	UQID
Reserved	1100b	Don't Care
Fast_GO_WritePull	1101b	UQID
GO_ERR_WritePull	1111b	UQID

1. 4-bit MESI encoding is in LSB and the upper bits are Reserved.

3.2.4.5.1 WritePull

This response instructs the device to send the write data to the Host, but not to change the state of the line. This is used for WrInv where the data is needed before the GO-I can be sent. This is because GO-I is the notification that the write was completed.

3.2.4.5.2 GO

The Global Observation (GO) message conveys that read requests are coherent and that write requests are coherent and consistent. It is an indication that the transaction has been observed by the system device and the MESI state that is encoded in the RspType field indicates into which state the data associated with the transaction should be placed for the requester's caches. Details in [Table 3-20](#).

If the Host returns Modified state to the device, then the device is responsible for the dirty data and cannot drop the line without writing it back to the Host.

If the Host returns Invalid or Error state to the device, then the device must use the data at most once and not cache the data. Error responses to reads and cacheable write requests (for example, RdOwn or ItoMWr) will always be the result of an abort condition, so modified data can be safely dropped in the device.

3.2.4.5.3 GO_WritePull

This is a combined GO + WritePull message. No cache state is transferred to the device. The GO+WritePull message is used for write types that do not require a later message to know whether write data is visible.

3.2.4.5.4 ExtCmp

This response indicates that the data that was previously locally ordered (FastGO) has been observed throughout the system. Most importantly, accesses to memory will return the most up-to-date data.

3.2.4.5.5 GO_WritePull_Drop

This message has the same semantics as GO_WritePull, except that the device should not send data to the Host. This response can be sent in place of GO_WritePull when the Host determines that the data is not required. This response will never be sent for partial writes because the byte enables will always need to be transferred.

3.2.4.5.6 Fast_GO_WritePull

Similar to GO_WritePull, but only indicates that the request is locally observed. There will be a later ExtCmp message when the transaction is fully observable in memory. Devices that do not implement the Fast_GO feature may ignore the GO message and wait for the ExtCMP. Data must always be sent for the WritePull. No cache state is transferred to the device.

Locally Observed, in this context, is a host-specific coherence domain that may be a subset of the global coherence domain. An example is a Last Level Cache that the requesting device shares with other CXL.cache devices that are connected below a host-bridge. In that example, local observation is only within the Last Level Cache and not between other Last Level Caches.

3.2.4.5.7 GO_ERR_WritePull

Similar to GO_WritePull, but indicates that there was an error with the transaction that should be handled correctly in the device. Data must be sent to the Host for the WritePull, and the Host will drop the data. No cache state is transferred to the device (assumed Error). An ExtCmp is still sent if it is expected by the originating request.

3.2.5 Cacheability Details and Request Restrictions

These details and restrictions apply to all devices.

3.2.5.1 GO-M Responses

GO-M responses from the host indicate that the device is being granted the sole copy of modified data. The device must cache this data and write it back when it is done.

3.2.5.2 Device/Host Snoop-GO-Data Assumptions

When the host returns a GO response to a device, the expectation is that a snoop arriving to the same address of the request receiving the GO would see the results of that GO. For example, if the host sends GO-E for an RdOwn request followed by a snoop to the same address immediately afterwards, then one would expect the device to transition the line to M state and reply with an RspIFwdM response back to the Host. To implement this principle, the CXL.cache link layer ensures that the device will receive the two messages in separate slots to make the order completely unambiguous.

When the host is sending a snoop to the device, the requirement is that no GO response will be sent to any requests with that address in the device until after the Host has received a response for the snoop and all implicit writeback (IWB) data (dirty data forwarded in response to a snoop) has been received.

When the host returns data to the device for a read type request, and GO for that request has not yet been sent to the device, the host may not send a snoop to that address until after the GO message has been sent. Because the new cache state is encoded in the response message for reads, sending a snoop to an address without having received GO, but after having received data, is ambiguous to the device as to what the snoop response should be in that situation.

Fundamentally, the GO that is associated with a read request also applies to the data returned with that request. Sending data for a read request implies that data is valid, meaning the device can consume it even if the GO has not yet arrived. The GO will arrive later and inform the device what state to cache the line in (if at all) and whether the data was the result of an error condition (e.g., hitting an address region that the device was not allowed to access).

3.2.5.3 Device/Host Snoop/WritePull Assumptions

The device requires that the host cannot have both a WritePull and H2D Snoop active on CXL.cache to a given 64-byte address. The host may not launch a snoop to a 64-byte address until all WritePull data from that address has been received by the host. Conversely, the host may not launch a WritePull for a write until the host has received the snoop response (including data in case of Rsp*Fwd*) for any snoops to the pending writes address. Any violation of these requirements will mean that the Bogus field on the D2H Data channel will be unreliable.

3.2.5.4 Snoop Responses and Data Transfer on CXL.cache Evicts

To snoop cache evictions (for example, DirtyEvict) and maintain an orderly transfer of snoop ownership from the device to the host, cache evictions on CXL.cache must adhere to the following protocol.

If a device Evict transaction has been issued on the CXL.cache D2H request channel, but has not yet processed its WritePull from the host, and a snoop hits the writeback, the device must track this snoop hit if cache state is changed, which excludes the case when SnpCur results in a RspVFwdV response. When the device begins to process the WritePull, if snoop hit is tracked the device must set the Bogus field in all the D2H data messages sent to the host. The intent is to communicate to the host that the request data was already sent as IWB data, so the data from the Evict is potentially stale.

3.2.5.5 Multiple Snoops to the Same Address

The host is only allowed to have one snoop pending at a time per cacheline address per device. The host must wait until it has received both the snoop response and all IWB data (if any) before sending the next snoop to that address.

3.2.5.6 Multiple Reads to the Same Cacheline

Multiple read requests (cacheable or uncacheable) to the same cacheline are allowed only in the following specific cases where host tracking state is consistent regardless of the order requests are processed. The host can freely reorder requests, so the device is responsible for ordering requests when required. For host memory, multiple RdCurr and/or CLFlush are allowed. For these commands the device ends in I-state, so there is no inconsistent state possible for host tracking of a device cache. With Type 2 devices that use HDM-D memory, in addition to RdCurr and/or CLFlush, multiple RdOwnNoData (bias flip requests) are allowed for device-attached memory. This case is allowed because with device-attached memory, the host does not track the device's cache so re-ordering in the host will not create an ambiguous state between the device and the host.

3.2.5.7 Multiple Evicts to the Same Cacheline

Multiple Evicts to the same cacheline are not allowed. All Evict messages from the device provide a guarantee to the host that the evicted cacheline will no longer be present in the device's caches.

Thus, it is a coherence violation to send another Evict for the same cacheline without an intervening cacheable Read/Read0 request to that address.

3.2.5.8 Multiple Write Requests to the Same Cacheline

Multiple WrInv/WOWrInv/ItoMWr/WrCur to the same cacheline are allowed to be outstanding on CXL.cache. The host or switch can freely reorder requests, and the device may receive corresponding H2D Responses in reordered manner. However, it is generally recommended that the device should issue no more than one outstanding Write request for a given cacheline, and order multiple write requests to the same cacheline one after another whenever stringent ordering is warranted.

3.2.5.9 Multiple Read and Write Requests to the Same Cacheline

Multiple RdCur/CLFlush/WrInv/WOWrInv/ItoMWr/WrCur may be issued in parallel from devices to the same cacheline address. Other reads need to issue one at a time (also known as "serialize"). To serialize, the read must not be issued until all other

outstanding accesses to same cacheline address have received GO. Additionally, after the serializing read is issued, no other accesses to the same cacheline address may be issued until it has received GO.

3.2.5.10 Normal Global Observation (GO)

Normal Global Observation (GO) responses are sent only after the host has guaranteed that request will have next ownership of the requested cacheline. GO messages for requests carry the cacheline state permitted through the MESI state or indicate that the data should only be used once and whether an error occurred.

3.2.5.11 Relaxed Global Observation (FastGO)

FastGO is only allowed for requests that do not require strict ordering. The Host may return the FastGO once the request is guaranteed next ownership of the requested cacheline within an implementation dependent sub-domain (e.g., CPU socket), but not necessarily within the system. Requests that receive a FastGO response and require completion messages are usually of the write combining memory type and the ordering requirement is that there will be a final completion (ExtCmp) message indicating that the request is at the stage where it is fully observed throughout the system. To make use of FastGO, devices have specific knowledge of the FastGO boundary of the CXL hierarchy and know the consumer of the data is within that hierarchy; otherwise, they must wait for the ExtCmp to know the data will be visible.

3.2.5.12 Evict to Device-attached Memory

Device Evicts to device-attached memory are not allowed on CXL.cache. Evictions are expected to go directly to the device's own memory; however, a device may use non-Evict writes (e.g., ItoMWr, WrCur) to write data to the host to device-attached memory.

3.2.5.13 Memory Type on CXL.cache

To source requests on CXL.cache, devices need to get the Host Physical Address (HPA) from the Host by means of an ATS request on CXL.io. Due to memory type restrictions, on the ATS completion, the Host indicates to the device if an HPA can only be issued on CXL.io as described in [Section 3.1.6](#). The device is not allowed to issue requests to such HPAs on CXL.cache. For requests that target ranges within the Device's local HDM range, the HPA is permitted to be obtained by means of an ATS request on CXL.io, or by using device-specific means.

3.2.5.14 General Assumptions

1. The Host will NOT preserve ordering of the CXL.cache requests as delivered by the device. The device must maintain the ordering of requests for the case(s) where ordering matters. For example, if D2H memory writes need to be ordered with respect to an MSI (on CXL.io), it is up to the device to implement the ordering. This is made possible by the non-posted nature of all requests on CXL.cache.
2. The order chosen by the Host will be conveyed differently for reads and writes. For reads, a Global Observation (GO) message conveys next ownership of the addressed cacheline; the data message conveys ordering with respect to other transactions. For writes, the GO message conveys both next ownership of the line and ordering with respect to other transactions.
3. The device may cache ownership and internally order writes to an address if a prior read to that address received either GO-E or GO-M.
4. For reads from the device, the Host transfers ownership of the cacheline with the GO message, even if the data response has not yet been received by the device. The device must respond to a snoop to a cacheline which has received GO, but if

- data from the current transaction is required (e.g., a RdOwn to write the line) the data portion of the snoop is delayed until the data response is received.
5. The Host must not send a snoop for an address where it has sent a data response for a previous read transaction but has not yet sent the GO. Ordering will ensure that the device observes the GO in this case before any later snoop. Refer to [Section 3.2.5.2](#) for additional details.
 6. Write requests (other than Evicts) such as WrInv, WOWrInv*, ItoMWr, and WrCur will never respond to WritePulls with data marked as Bogus.
 7. The Host must not send two cacheline data responses to the same device request. The device may assume one-time use ownership (based on the request) and begin processing for any part of a cacheline received by the device before the GO message. Final state information will arrive with the GO message, at which time the device can either cache the line or drop it depending on the response.
 8. For a given transaction, H2D Data transfers must come in consecutive packets in natural order with no interleaved transfers from other lines.
 9. D2H Data transfer of a cacheline must come in consecutive packets with no interleaved transfers from other lines. The data must come in natural chunk order, that is, 64B transfers must complete the lower 32B half first because snoops are always cacheline aligned.
 10. Device snoop responses in D2H Response must not be dependent on any other channel or on any other requests in the device besides the availability of credits in the D2H Response channel. The Host must guarantee that the responses will eventually be serviced and return credits to the device.
 11. The Host must not send a second snoop request to an address until all responses, plus data if required, for the prior snoop are collected.
 12. H2D Response and H2D Data messages to the device must drain without the need for any other transaction to make progress.
 13. The Host must not return GO-M for data that is not actually modified with respect to memory.
 14. The Host must not write unmodified data back to memory.
 15. Except for WOWrInv and WOWrInF, all other writes are strongly ordered

3.2.5.15 Buried Cache State Rules

Buried Cache state refers to the state of the cacheline registered in the Device's Coherency engine (DCOH) when a CXL.cache request is being sent for that cacheline from the device.

The Buried Cache state rules for a device when issuing CXL.cache requests are as follows:

- Must not issue a Read if the cacheline is buried in Modified, Exclusive, or Shared state.
- Must not issue RdOwnNoData if the cacheline is buried in Modified or Exclusive state. The Device may request for ownership in Exclusive state as an upgrade request from Shared state.
- Must not issue a Read0-Write if the cacheline is buried in Modified, Exclusive, or Shared state.
- All *Evict opcodes must adhere to apropos use case. For example, the Device is allowed to issue DirtyEvict for a cacheline only when it is buried in Modified state. For performance benefits, it is recommended that the Device should not silently drop a cacheline in Exclusive or Shared state and instead use CleanEvict* opcodes toward the Host.

- The CacheFlushed Opcode is not specific to a cacheline, it is an indication to the Host that all the Device's caches are flushed. Thus, the Device must not issue CacheFlushed if there is any cacheline buried in Modified, Exclusive, or Shared state.

Table 3-28 describes which Opcodes in D2H requests are allowed for a given Buried Cache State.

IMPLEMENTATION NOTE

Buried state rules are requirements at the requester's Transaction Layer. It is possible snoops in flight to change the state observed at the host before the host processes the request. An example case is SnpInv sent from the host at the same time as the device issues a CleanEvictNoData from E-state, the snoop will cause the cache state in the device to change to I-state before the CleanEvictNoData is processed in the host, so the host must allow for this degraded cache state in its coherence tracking.

Table 3-28. Allowed Opcodes for D2H Requests per Buried Cache State

D2H Requests		Buried Cache State			
Opcodes	Semantic	Modified	Exclusive	Shared	Invalid
RdCurr	Read				X
RdOwn	Read				X
RdShared	Read				X
RdAny	Read				X
RdOwnNoData	Read0			X	X
ItoMWr	Read0-Write				X
WrCur	Read0-Write				X
CLFlush	Read0				X
CleanEvict	Write		X		
DirtyEvict	Write	X			
CleanEvictNoData	Write		X	X	
WOWrInv	Write				X
WOWrInvF	Write				X
WrInv	Write				X
CacheFlushed	Read0				X

3.2.5.16 H2D Req Targeting Device-attached Memory

H2D Req messages are sent by a host to a device because the host believes that the device may own a cacheline that the device previously received from this same host. The very principle of a Type 2 Device is to provide direct access to Device-attached Memory (i.e., without going through its host). Host coherence for this region is managed by using the M2S Req channel. These statements combined could lead a Type 2 Device to assume that H2D Req messages can never target addresses that belong to the Device-attached memory by design.

However, a host may decide to snoop more cache peers than strictly required, without any other consideration than the cache peer being visible to the host. This type of behavior is allowed by the CXL protocol and can occur for multiple reasons, including coarse tracking and proprietary RAS features. In that context, a host may generate an

H2D Req to a Type 2 device on addresses that belong to the Device-attached Memory. An H2D Req from the host that targets Device-attached memory can cause coherency issues if the device were to respond with data and, more generally speaking, protocol corner cases.

To avoid these issues, both HDM-D Type 2 devices and HDM-DB Type 2 devices are required to:

- Detect H2D Req that target Device-attached Memory
- When detected, unconditionally respond with RspIHitI, disregarding all internal states and without changing any internal states (e.g., don't touch the cache)

3.3

CXL.mem

3.3.1

Introduction

The CXL Memory Protocol is called CXL.mem, and it is a transactional interface between the CPU and Memory. It uses the phy and link layer of CXL when communicating across dies. The protocol can be used for multiple different Memory attach options including when the Memory Controller is located in the Host CPU, when the Memory Controller is within an Accelerator device, or when the Memory Controller is moved to a memory buffer chip. It applies to different Memory types (e.g., volatile, persistent, etc.) and configurations (e.g., flat, hierarchical, etc.) as well.

The CXL.mem provides 3 basic coherence models for CXL.mem Host-managed Device Memory (HDM) address regions exposed by the CXL.mem protocol:

- HDM-H (Host-only Coherent): Used only for Type 3 Devices
- HDM-D (Device Coherent): Used only for legacy Type 2 Devices that rely on CXL.cache to manage coherence with the Host
- HDM-DB (Device Coherent using Back-Invalidate): Can be used by Type 2 Devices or Type 3 Devices

Note:

The view of the address region must be consistent on the CXL.mem path between the Host and the Device.

The coherency engine in the CPU interfaces with the Memory (Mem) using CXL.mem requests and responses. In this configuration, the CPU coherency engine is regarded as the CXL.mem Master and the Mem device is regarded as the CXL.mem Subordinate. The CXL.mem Master is the agent which is responsible for sourcing CXL.mem requests (e.g., reads, writes, etc.) and a CXL.mem Subordinate is the agent which is responsible for responding to CXL.mem requests (e.g., data, completions, etc.).

When the Subordinate maps HDM-D/HDM-DB, CXL.mem protocol assumes the presence of a device coherency engine (DCOH). This agent is assumed to be responsible for implementing coherency related functions such as snooping of device caches based on CXL.mem commands and update of Metadata fields.

Support for memory with Metadata is optional but this needs to be negotiated with the Host in advance. If the device supports "Metabits Storage" Feature, this mechanism may be used to negotiate the Metadata configuration. Other negotiation mechanisms are beyond the scope of this specification. If Metadata is not supported by device-attached memory, the DCOH will still need to use the Host supplied Metadata updates to interpret the commands. If Metadata is supported by device-attached memory, it can be used by Host to implement a coarse snoop filter for CPU sockets. In the HDM-H address region, the usage is defined by the Host. The protocol allows for 2 bits of Metadata to be stored and returned.